Priloga 1

ŠOLSKI CENTER VELENJE

ELEKTRO IN RAČUNALNIŠKA ŠOLA

IZDELEK OZ. STORITEV ZA 4. PREDMET POKLICNE MATURE

PISNA DOKUMENTACIJA

Velenje, januar 2026

Tej Verbič, Andraž Dimec

Priloga 2

ŠOLSKI CENTER VELENJE
ELEKTRO IN RAČUNALNIŠKA ŠOLA
Trg mladosti 3
3320 VELENJE

IZDELEK OZ. STORITEV PO IZOBRAŽEVALNEM PROGRAMU
SREDNJEGA STROKOVNEGA IZOBRAŽEVANJA
TEHNIK RAČUNALNIŠTVA

ŠOLSKA KNJIŽNICA IN IZPOSOJA TEHNIČNE OPREME

Avtorja: Tej Verbič, Andraž Dimec
Mentor na šoli: Islam Mušić

POVZETEK

Cilj najine naloge je bil razviti celovito spletno aplikacijo za integrirano upravljanje šolske knjižnice in izposojo tehnične opreme. Aplikacija omogoča avtomatizacijo procesov vodenja inventarja, registracijo dijakov ter dodajanje, urejanje in pregledovanje gradiva ter opreme. Sistem vključuje podrobne opise predmetov, sledenje razpoložljivosti zalog, napreden rezervacijski sistem ter digitalno vodenje evidenc o poškodbah s slikovnim gradivom.

Projekt je zasnovan kot moderna spletna aplikacija z ločenim zalednim delom (backend), razvitim v ogrodju .NET 9.0, in čelnim delom (frontend), razvitim v knjižnici React 19. Podatkovna baza PostgreSQL zagotavlja varno in zanesljivo shranjevanje podatkov, komunikacija med deli sistema pa poteka preko standardiziranih REST API-jev. Varnost aplikacije je zagotovljena s kriptografskim zgoščevanjem gesel z algoritmom BCrypt in uporabo naprednega preslikovanja Entity Framework Core, ki preprečuje nepooblaščene posege v podatke.

Rezultat najinega projekta je popolnoma funkcionalna spletna rešitev, ki omogoča enostavno digitalizacijo šolskih knjižnic in predstavlja uporabno orodje za izboljšanje sledljivosti šolske lastnine ter optimizacijo izposoje tehnične opreme.

Kazalo vsebine

ŠOLSKA KNJIŽNICA IN IZPOSOJA TEHNIČNE OPREME 2
POVZETEK 3
SEZNAM OKRAJŠAV 6
1. UVOD 7
1.1 Opis področja in problema 7
1.2 Namen in cilji naloge 7
1.3 Metode dela 8
2. TEORETIČNA IZHODIŠČA 10
2.1 Uporabljene tehnologije 10
2.1.1 .NET 9.0 (ASP.NET Core) 10
2.1.2 React 19 (s TypeScript in Vite) 10
2.1.3 PostgreSQL 11
2.1.4 REST API 11
2.1.5 Tehnične zahteve in orodja 11
2.2 Arhitektura aplikacije 13
2.3 Varnost in avtentikacija 13
3. RAZVOJ IN IZDELAVA 14
3.1 Načrtovanje rešitve 14
3.2 Podatkovna baza 15
3.2.2 Opis entitet in relacij 15
3.3 Zaledni del (Backend) 16
3.3.1 Kontrolerji 16
3.3.2 Modeli in Entity Framework Core 16
3.3.3 Načrt REST API končnih točk 17
3.4 Čelni del (Frontend) 19
3.4.1 Struktura aplikacije 19
3.4.2 Ključne strani 19
3.4.3 Opis navigacije 19
3.4.4 Storitve in API komunikacija 20
3.4.5 Postopek razvoja (Organizacija dela) 21
4. ANALIZA 22
4.1 Testiranje 22
4.2 Predstavitev končnega izdelka 22
4.3 Rezultati 23
5. ZAKLJUČEK 24

5.1 Samoevalvacija 25

SEZNAM OKRAJŠAV

API Application Programming Interface / Programski vmesnik aplikacije

SPA Single Page Application / Enostranska spletna aplikacija

ORM Object-Relational Mapping / Objektno-relacijsko preslikovanje

UX User Experience / Uporabniška izkušnja

UI User Interface / Uporabniški vmesnik

CRUD Create, Read, Update, Delete / Operacije ustvari, preberi, posodobi, izbriši

JSON JavaScript Object Notation / Format za izmenjavo podatkov

HTTP HyperText Transfer Protocol / Protokol za prenos hiperteksta

SDK Software Development Kit / Razvojni paket programske opreme

EF Core Entity Framework Core / Microsoftovo orodje za delo s podatkovnimi bazami

CSS Cascading Style Sheets / Kaskadne slogovne predloge

PDF Portable Document Format / Prenosljiv format dokumenta

DB Database / Podatkovna baza

URL Uniform Resource Locator / Spletni naslov

1. UVOD

1.1 Opis področja in problema

V izobraževalnih ustanovah se pogosto srečujeva z izzivom učinkovitega upravljanja z viri. Tradicionalne šolske knjižnice so omejene le na papirno gradivo, medtem ko se tehnična oprema pogosto vodi ločeno, kar povzroča informacijski razkol. Šolski administratorji težko sledijo zasedenosti opreme, stanju (poškodbam) in odgovornosti posameznega dijaka.

Problem, ki ga aplikacija rešuje, je digitalizacija in poenotenje teh procesov. Sistem omogoča, da se knjige in tehnična oprema obravnavajo kot predmeti izposoje znotraj enovite platforme, kar omogoča boljši nadzor nad zalogami, hitrejšo izposojo in transparentno beleženje poškodb.

1.2 Namen in cilji naloge

Namen zaključne naloge je razviti celovito in varnostno podprto spletno aplikacijo za upravljanje šolskega inventarja, ki bo omogočala:

• avtentikacijo uporabnikov: registracijo in varno prijavo z različnimi ravnmi dostopa (administrator in dijak),
• upravljanje inventarja: dodajanje novih knjig in tehnične opreme (npr. prenosni računalniki, tablice) z natančnimi tehničnimi opisi,
• avtomatizacijo CRUD operacij: urejanje, posodabljanje zalog in brisanje obstoječih predmetov iz kataloga,
• napredno iskanje in filtriranje: pregledovanje knjižničnega gradiva in opreme glede na kategorije, jezike ali status razpoložljivosti,
• sledenje izposojam in rezervacijam: vodenje digitalne evidence o tem, kdo ima kateri predmet in do kdaj,
• dokumentiranje poškodb: povezovanje izposoj s poročili o poškodbah, vključno s shranjevanjem slikovnega gradiva,
• dostopnost: omogočanje dostopa do aplikacije preko spletnega brskalnika in dokumentiranih REST API-jev.

Cilji naloge so:

1. Razvoj zalednega dela (Backend): Izdelati robusten strežniški del aplikacije v ogrodju .NET 9.0 z uporabo arhitekture REST API za učinkovito izmenjavo podatkov.
2. Razvoj čelnega dela (Frontend): Izdelati moderen in odziven uporabniški vmesnik v ogrodju React 19 z uporabo TypeScript-a za boljšo stabilnost kode.
3. Implementacija podatkovne baze: Vzpostaviti PostgreSQL podatkovno bazo in uporabiti Entity Framework Core za napredno upravljanje relacij med entitetami.
4. Zagotavljanje varnosti: Implementirati varno shranjevanje gesel z algoritmom BCrypt ter zaščititi API točke pred nepooblaščenimi dostopi.
5. Gostovanje aplikacije: Uspešno objaviti in konfigurirati aplikacijo na platformi Render za javno dostopnost.
6. Dokumentacija in verzioniranje: Dokumentirati celoten postopek razvoja (načrtovanje, razvoj, analiza) ter omogočiti dostop do izvorne kode preko repozitorija na platformi GitHub.

1.3 Metode dela

Pri razvoju aplikacije sva uporabila naslednje metode in strokovne pristope:

• Načrtovanje: Najprej sva opravila analizo zahtev šolskega okolja. Načrtovala sva strukturo relacijske podatkovne baze in arhitekturo aplikacije v orodju Figma, kjer sva izdelala žične modele (wireframes) za ključne strani uporabniškega vmesnika. Uporabila sva diagrame povezav med entitetami (ERD) za definiranje odnosov med knjigami, opremo in dijaki.

• Razvoj backenda: Zaledni del sva razvila v programskem jeziku C# z uporabo ogrodja .NET 9.0. Uporabila sva Entity Framework Core (EF Core) za objektno-relacijsko preslikavo in komunikacijo s podatkovno bazo. Implementirala sva REST API končne točke, ki omogočajo varno izmenjavo podatkov v JSON formatu.

• Razvoj frontenda: Čelni del sva razvila v knjižnici React 19 z uporabo TypeScript-a za strožjo tipizacijo podatkov. Za oblikovanje modernega in odzivnega uporabniškega vmesnika s steklenim učinkom (glassmorphism) sva uporabila Tailwind CSS 4. Za asinhrono komunikacijo z API-jem sva uporabila knjižnico Axios.

• Podatkovna baza: Za shranjevanje podatkov sva uporabila PostgreSQL. Inicializacijo sheme (ustvarjanje tabel in relacij) sva izvedla programsko znotraj zalednega dela aplikacije preko SQL skript, kar zagotavlja enostavno namestitev v nova okolja.

• Testiranje: Aplikacijo sva testirala z metodo »črne skrinje« (black-box testing) z uporabo različnih realnih scenarijev: od registracije novega dijaka do kompleksnih rezervacij opreme v različnih terminih. API točke sva preverjala preko vgrajenega orodja Swagger.

• Gostovanje: Aplikacijo sva pripravila za produkcijsko okolje in jo gostovala na platformi Render, ki omogoča avtomatizirano osveževanje aplikacije ob vsaki spremembi v kodi (CI/CD proces).

2. TEORETIČNA IZHODIŠČA

2.1 Uporabljene tehnologije

2.1.1 .NET 9.0 (ASP.NET Core)

.NET 9.0 je najsodobnejše odprtokodno ogrodje podjetja Microsoft za razvoj visokozmogljivih spletnih in oblačnih aplikacij. ASP.NET Core znotraj tega ogrodja poenostavlja razvoj varnih in skalabilnih RESTful API-jev.

Glavne prednosti .NET 9.0:

• Visoka zmogljivost: Velja za eno najhitrejših ogrodij za razvoj spletnih servisov.

• Entity Framework Core: Močno orodje za objektno-relacijsko preslikavo (ORM), ki poenostavlja delo z bazo podatkov preko C# razredov.

• Vgrajena podpora za Dependency Injection: Omogoča boljšo organizacijo kode in lažje testiranje.

• Varnost: Vključuje robustne mehanizme za avtentikacijo, avtorizacijo in zaščito pred spletnimi ranljivostmi.

V projektu sva uporabila verzijo .NET 9.0 s programskim jezikom C# 13.

2.1.2 React 19 (s TypeScript in Vite)

React je knjižnica za razvoj dinamičnih uporabniških vmesnikov, razvita s strani podjetja Meta. Verzija 19 prinaša izboljšano upravljanje s stanji in optimizirano upodobitev komponent.

Glavne prednosti React:

• Komponentna arhitektura: Omogoča razdelitev vmesnika na manjše, ponovno uporabljive gradnike.

• Vite razvojno okolje: Zagotavlja izjemno hitro prevajanje kode in takojšen vpogled v spremembe med razvojem.

• TypeScript podpora: Programiranje s strogo določenimi tipi podatkov zmanjšuje število logičnih napak.

• Virtual DOM: Optimizira posodabljanje zaslona, kar zagotavlja tekočo uporabniško izkušnjo.

V projektu sva uporabila React 19 v kombinaciji s Tailwind CSS 4 za hitro oblikovanje vmesnika.

2.1.3 PostgreSQL

PostgreSQL je eden najnaprednejših odprtokodnih relacijskih sistemov za upravljanje baz podatkov. Podpira napredne tipe podatkov in zagotavlja visoko stopnjo integritete podatkov.

Glavne prednosti PostgreSQL:

• ACID skladnost: Zagotavlja zanesljivost transakcij tudi ob morebitnih sistemskih napakah.

• Podpora za kompleksne relacije: Odlično obvladuje Many-to-Many in One-to-Many povezave med tabelami.

• Varnost: Močna podpora za SSL šifriranje povezav in napredno upravljanje dostopa.

Infrastruktura baze v projektu temelji na PostgreSQL, ki gostuje v oblaku na platformi Supabase.

2.1.4 REST API

REST (Representational State Transfer) je arhitekturni slog za razvoj spletnih storitev, ki omogoča komunikacijo med frontendom in backendom preko HTTP protokola.

Glavne značilnosti REST API:

• Uporaba standardnih metod: GET (branje), POST (ustvarjanje), PUT (posodabljanje) in DELETE (brisanje).

• Brezstanjskost: Strežnik ne shranjuje stanja odjemalca, kar povečuje skalabilnost.

• JSON format: Podatki se prenašajo v lahki in berljivi obliki, primerni za JavaScript.

V projektu sva implementirala REST API končne točke za vse operacije nad knjigami, opremo, izposojami in rezervacijami.

2.1.5 Tehnične zahteve in orodja

Tehnične zahteve za delovanje:

- Strežniški del (Backend): Zahteva nameščen .NET 9.0 SDK. Aplikacija teče na Kestrel strežniku, za shranjevanje podatkov pa potrebuje dostop do PostgreSQL baze (lokalno ali v oblaku).
- Odjemalec (Frontend): Zahteva okolje Node.js (različica 18 ali novejša) za namestitev odvisnosti in poganjanje razvojnega strežnika. Končni uporabnik potrebuje le sodoben spletni brskalnik (Chrome, Edge, Firefox).

Uporabljena orodja:

- Razvojno okolje (IDE): Visual Studio Code (z razširitvami za C# in React).
- Načrtovanje: Figma (za načrtovanje vmesnika) in dbdiagram.io (za načrtovanje E-R modela).
- Verzioniranje: Git in storitev GitHub za usklajevanje dela med razvijalcema.
- Testiranje API-ja: Swagger UI (vgrajen v .NET) in Postman.

Zunanje komponente (Libraries):

- BCrypt.Net-Next: Za varno zgoščevanje gesel.
- Npgsql.EntityFrameworkCore.PostgreSQL: Gonilnik za povezavo s PostgreSQL.
- Axios: Za HTTP zahtevke na frontendu.
- Framer Motion: Za tekoče animacije v uporabniškem vmesniku.
- Lucide React: Za ikone v aplikaciji.

2.2 Arhitektura aplikacije

Aplikacija temelji na arhitekturi odjemalec-strežnik (client-server), kjer sta uporabniški vmesnik in poslovna logika popolnoma ločena.

Zaledni del (Backend):

• Razvit v .NET 9.0.

• Vsebuje kontrolerje (vhodne točke), modele (entitete) in podatkovni kontekst (DbContext).

• Komunicira s PostgreSQL bazo preko Entity Framework Core.

• Izpostavlja varno OpenAPI/Swagger dokumentacijo.

Čelni del (Frontend):

• Razvit v React 19.

• Vsebuje komponente, strani in servise za klice API.

• Komunicira z zalednim delom preko knjižnice Axios.

• Prikazuje podatke na odziven in interaktiven način.

Podatkovna baza:

• Relacijski PostgreSQL model.

• Vsebuje tabele za uporabnike, gradivo in evidenco izposoj.

• Zagotavlja trajno in varno shranjevanje vseh zapisov.

2.3 Varnost in avtentikacija

Varnost aplikacije je zagotovljena na več nivojih:

• Šifriranje gesel: Vsa uporabniška gesla so zgoščena (hashed) z uporabo algoritma BCrypt.Net. To zagotavlja, da gesla niso berljiva niti administratorjem baze.

• Preprečevanje SQL injekcij: Uporaba Entity Framework Core samodejno preprečuje SQL injection napade, saj se vse vrednosti v podatkovno bazo pošiljajo kot varni parametri.

• CORS konfiguracija: Strežnik ima nastavljeno politiko Cross-Origin Resource Sharing, ki dovoljuje dostop do API-ja le pooblaščenim domenam (npr. frontend aplikaciji).

• Varen prenos podatkov: Povezava s podatkovno bazo v oblaku uporablja SSL šifriranje, kar preprečuje prisluškovanje podatkom med prenosom.

3. RAZVOJ IN IZDELAVA

3.1 Načrtovanje rešitve

Pred začetkom razvoja sva analizirala delovne procese v šolski knjižnici in določila strukturo podatkovne baze. Opredelila sva naslednje glavne entitete:

• Uporabniki: Administratorji in knjižničarji, ki upravljajo sistem.

• Dijaki: Uporabniki, ki si izposojajo gradivo in opremo.

• Knjige: Knjižnično gradivo s podatki o naslovu, avtorju in jeziku.

• Oprema: Tehnični predmeti (prenosniki, projektorji) s serijskimi številkami.

• Izposoje: Zapisi o trenutnih in preteklih izposojah predmetov.

• Rezervacije: Načrtovane izposoje za prihodnja obdobja.

• Poškodbe: Evidenca poškodovanega inventarja s slikovnimi dokazi.

Načrtovane relacije:

• Mnogo-proti-mnogo (Many-to-Many): Knjige in avtorji so povezani preko vmesne tabele *avtor_knjiga*.

• Ena-proti-mnogo (One-to-Many): Ena kategorija vsebuje več kosov opreme; en jezik pripada več knjigam; en dijak ima lahko več izposoj.

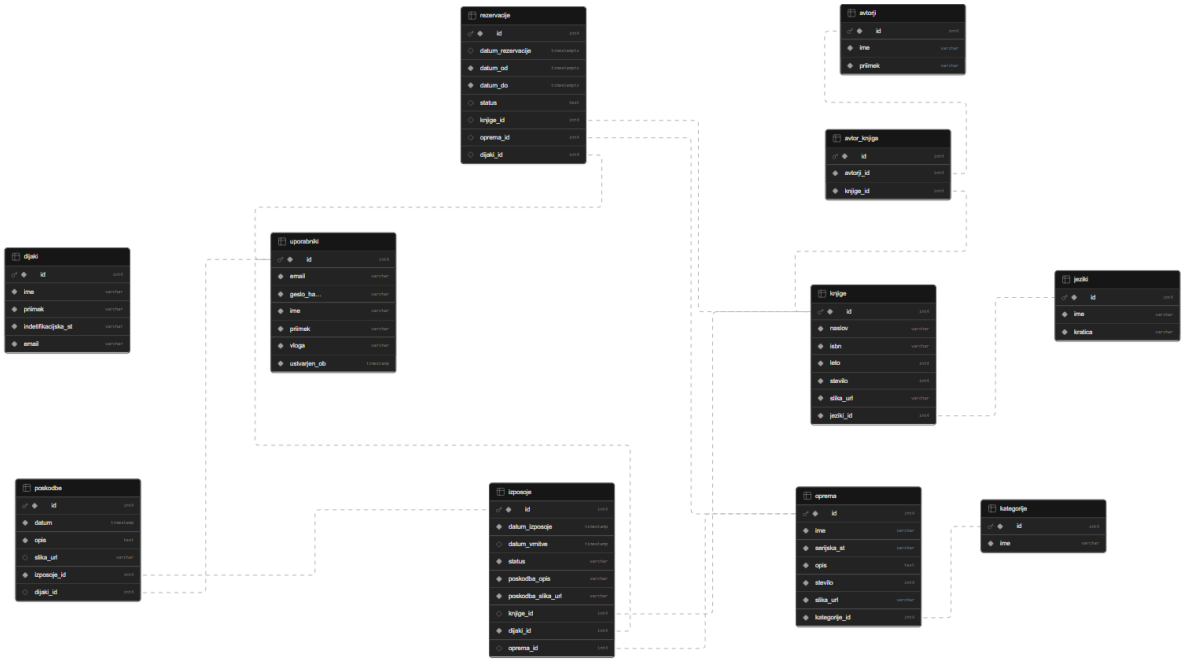
3.2 Podatkovna baza

Pri načrtovanju baze podatkov sva se osredotočila na visoko stopnjo normalizacije, s katero sva preprečila podvajanje podatkov in zagotovila integriteto relacij. Za shranjevanje sva uporabila relacijski sistem PostgreSQL.

3.2.1 E-R model (grafični prikaz)

Najin E-R model (Entity-Relationship) predstavlja logično strukturo baze. Sestavljajo ga entitete (tabele), atributi (stolpci) in povezave (ključne relacije).

- **Entitete:** Predstavljajo glavne objekte v sistemu (npr. Knjige, Dijaki, Oprema).
- **Atributi:** Vsaka entiteta ima določen primarni ključ (id) in specifične lastnosti (npr. naslov knjige, serijska številka opreme).
- **Povezave:** Definirane so s tujimi ključi, ki povezujejo tabele (npr. povezava med knjigo in njenim jezikom).



3.2.2 Opis entitet in relacij

Spodaj podrobneje opisujeva ključne tabele najine baze podatkov in njihove medsebojne vloge:

1. **Uporabniki (uporabniki):** Centralna tabela za avtentikacijo. Shranjuje podatke o administratorjih in dijakih, vključno z zgoščenim geslom (geslo_hash) in vlogo (vloga), ki določa pravice v aplikaciji.
2. **Katalog gradiva (knjige in oprema):**
    - Tabela knjige je povezana s tabelo jeziki (relacija Many-to-One), kar omogoča standardiziran vnos jezikov.
    - Tabela oprema je povezana s tabelo kategorije (npr. prenosniki, projektorji).

3. **Avtorji in Knjige (avtorji, avtor_knjiga):** Ker ima lahko ena knjiga več avtorjev in en avtor več knjig, sva implementirala vmesno tabelo avtor_knjiga. Ta realizira relacijo **mnogo-proti-mnogo (Many-to-Many)**.
4. **Izposoje in Rezervacije (izposoje, rezervacije):** To sta ključni procesni tabeli.
    - Obe sta povezani z uporabniki (tuji ključ dijaki_id kaže na tabelo uporabniki).
    - Povezani sta neobvezno s knjigami ali opremo (polje je lahko NULL, če si dijak izposodi le eno vrsto gradiva).
    - Vključujeta časovne žige (datum_izposoje, datum_od, datum_do) za sledenje rokom.
5. **Beleženje poškodb (poskodbe):** Ta entiteta omogoča dokumentiranje nepravilnosti ob vračilu. Povezana je neposredno z določeno izposojo (izposoje_id) in uporabnikom, ki je škodo povzročil. Vključuje tudi polje slika_url za povezavo do fotografije poškodbe.

Relacijska pravila: Za zagotavljanje varnosti sva uporabila omejitve tujih ključev (FOREIGN KEY s CONSTRAINT), ki preprečujejo brisanje zapisov, če so ti še vedno povezani z aktivnimi izposojami ali rezervacijami. S tem sva zagotovila, da zgodovina izposoj ostane varno shranjena v bazi.

3.3 Zaledni del (Backend)

3.3.1 Kontrolerji

Kontrolerji v .NET 9.0 skrbijo za sprejemanje HTTP zahtevkov. Implementirala sva naslednje ključne kontrolerje:

• AuthController: Skrbi za /api/auth/register in /api/auth/login.

• KnjigeController: Omogoča CRUD operacije nad knjigami.

• OpremaController: Upravljanje s tehnično opremo.

• LoansController: Logika za izposojo in proces vračila predmetov.

• ReservationsController: Upravljanje rezervacij za vnaprej določene termine.

• UploadController: Namenska točka za varno nalaganje slik poškodb in inventarja.

3.3.2 Modeli in Entity Framework Core

Uporabila sva ApplicationDbContext za upravljanje z bazo. Vsaka tabela ima ustrezno C# entiteto.

Primer modela Knjiga:

```
namespace SolskaKnjiznica.Core.Entities;

public class Knjige
{
    public int Id { get; set; }
    public string Naslov { get; set; } = string.Empty;
```

```
    public string Isbn { get; set; } = string.Empty;
    public int Leto { get; set; }
    public int Stevilo { get; set; }
    public string SlikaUrl { get; set; } = string.Empty;
    public int JezikiId { get; set; }
}
```

3.3.3 Načrt REST API končnih točk

REST API predstavlja osrednji komunikacijski most najine aplikacije. Za izmenjavo podatkov uporablja format **JSON**. Spodaj so navedene ključne končne točke (endpoints), razvrščene po kontrolerjih:

Link do swagger UIja: https://knjiznica-backend.onrender.com/swagger

1. Avtentikacija (/api/auth)

Metoda	Pot	Opis
POST	/api/auth/register	Registracija novega uporabnika/administratorja.
POST	/api/auth/login	Prijava uporabnika in pridobitev podatkov o vlogi.

2. Upravljanje knjig in opreme (/api/knjige, /api/oprema, /api/items)

Metoda	Pot	Opis
GET	/api/items	Pridobitev združenega seznama vseh predmetov za izposojo.
GET	/api/knjige	Pridobitev seznama vseh knjig.
GET	/api/knjige/{id}	Pridobitev podrobnosti o specifični knjigi po ID-ju.
POST	/api/knjige	Dodajanje nove knjige v katalog (Admin).
PUT	/api/knjige/{id}	Posodabljanje podatkov o obstoječi knjigi (Admin).
DELETE	/api/knjige/{id}	Brisanje knjige iz sistema (Admin).

Metoda	Pot	Opis
GET	/api/oprema	Pridobitev seznama vse tehnične opreme.
POST	/api/oprema	Dodajanje novega kosa tehnične opreme (Admin).

3. Izposoje in Rezervacije (/api/loans, /api/reservations)

Metoda	Pot	Opis
GET	/api/loans	Pridobitev seznama vseh aktivnih in preteklih izposoj.
PUT	/api/loans/{id}/return	Zaključek izposoje (vračilo) z možnostjo vnosa poškodb.
GET	/api/reservations	Pridobitev seznama vseh opravljenih rezervacij.
POST	/api/reservations	Ustvarjanje nove rezervacije za določen termin.
PUT	/api/reservations/{id}/status	Sprememba statusa rezervacije (npr. iz "aktivna" v "preklicana").

4. Sistemske storitve (/api/jeziki, /api/kategorije, /api/upload)

Metoda	Pot	Opis
GET	/api/jeziki	Pridobitev seznama podprtih jezikov za knjige.
GET	/api/kategorije	Pridobitev seznama kategorij opreme (npr. kamere, prenosniki).
POST	/api/upload/image	Varno nalaganje slik na strežnik in pridobitev URL naslova.

Tehnične podrobnosti odgovorov: Vsak API klic vrne ustrezno HTTP statusno kodo:

- 200 OK: Zahteva je bila uspešno obdelana.
- 201 Created: Nov vir je bil uspešno ustvarjen.

- 400 Bad Request: Napaka v podatkih (npr. manjka obvezno polje).
- 401 Unauthorized: Uporabnik ni prijavljen ali nima pravic.
- 404 Not Found: Iskan predmet ali pot ne obstaja.
- 500 Internal Server Error: Napaka na strani strežnika.

3.4 Čelni del (Frontend)

3.4.1 Struktura aplikacije

Uporabniški vmesnik je zgrajen modularno v ogrodju React 19:

• Strani (pages): Prijava, Registracija, Nadzorna plošča, Katalog gradiva, Seznam rezervacij, Profil uporabnika.
• Komponente (components): Navigacija, Kartica predmeta (ItemCard), Obrazec za beleženje poškodb, Modalna okna za potrditve.
• Servisi (services): apiService.ts – vsebuje vso logiko za komunikacijo s strežnikom.

3.4.2 Ključne strani

• Dashboard: Prikazuje grafične povzetke o prostih knjigah in zasedeni opremi.
• Inventory: Omogoča iskanje in filtriranje po kategorijah opreme (npr. »Prenosniki«) ali jeziku knjig.
• Rezervacijski obrazec: Uporabnik izbere obdobje (od-do) in preveri razpoložljivost v realnem času.

3.4.3 Opis navigacije

1. Navigacija

- **Sistem SPA (Single Page Application):** Preklapljanje med stranmi poteka takoj, brez osveževanja brskalnika (uporaba react-router-dom).
- **Glavni meni (Header):** Vedno viden na vrhu. Vsebuje logotip, gumb za odjavo in gumb **"Dodaj predmet"** (samo za knjižničarje).
- **Sistem zavihkov (Tabs):** Na glavni plošči knjižničar z enim klikom preklaplja med Izposojami, Rezervacijami, Knjigami in Opremo.
- **Dinamične poti:**

- /login – Vstopna stran.
- /librarian – Glavna plošča za knjižničarja.
- /librarian/dodaj-predmet – Obrazec za vnos novega inventarja.
- /dashboard – Pregled za dijake.

2. Vsebina

- **Zavihek Izposoje:** Tabela z aktivnimi izposojami (kdo ima kaj, datum izposoje/vrnitve) in gumbom "Vrni" za zaključek izposoje.
- **Zavihek Rezervacije:** Seznam zahtev dijakov, kjer lahko knjižničar rezervacijo potrdi ali zavrne.
- **Zavihek Knjige/Oprema:** Vizualna mreža kartic (Grid). Vsaka kartica ima sliko predmeta, naslov, inventarno številko in gumbe za urejanje or brisanje.
- **Obrazec za dodajanje:** Polja za naslov, opis, kategorijo, količino in poseben gumb za nalaganje slike predmeta.
- **Dialogi (Pop-ups):** Obrazci, ki se pojavijo čez zaslon za hitro urejanje podatkov ali potrjevanje izbrisov.

3.4.4 Storitve in API komunikacija

Uporabila sva knjižnico Axios za izvajanje asinhronih zahtevkov.

Primer metode v apiService.ts:

```
getBook: async (id: number): Promise<any> => {
  const response = await api.get(`/knjige/${id}`);
  return response.data;
},
```

Vračilo predmeta z opcijskim zapisom poškodbe

```
returnItem: async (id: number, returnData: any): Promise<any> => {
  const response = await api.put(`/loans/${id}/return`, returnData);
  return response.data;
```

```
}
```

3.4.5 Postopek razvoja (Organizacija dela)

Pri razvoju projekta sva si delo organizirala s pomočjo orodja Trello, kjer sva ustvarila seznam nalog (ang. task board), razdeljen na posamezne funkcionalne sklope aplikacije. Vsaka naloga je predstavljala celotno vertikalo funkcionalnosti (baza podatkov, .NET API končne točke in React komponenta), kar je zagotavljalo dosledno arhitekturo po načelu full-stack implementacije.

Delo sva si razdelila tako, da je vsak član samostojno razvil posamezne funkcionalnosti od baze do uporabniškega vmesnika. Andraž Dimec je implementiral npr. vertikale za upravljanje izposoj, knjig in opreme, medtem ko je Tej Verbič razvil vertikale za rezervacijski sistem, prijavo uporabnikov in povezavo z bazo podatkov. Pri zahtevnejših delih, kot je usklajevanje API-ja z React komponentami in strukturo podatkov, sva sodelovala in sproti usklajevala rešitve.

Napredek sva redno spremljala preko Trella, kar nama je omogočilo pregledno sledenje nalogam, dobro organizacijo dela in uspešno implementacijo vseh funkcionalnosti v skladu z zahtevami projekta.

4. ANALIZA

4.1 Testiranje

Aplikacijo sva testirala ročno z uporabo različnih scenarijev uporabe, da bi zagotovila stabilnost in pravilno delovanje vseh poslovnih procesov.

Registracija in prijava:

• Registracija novega administratorja z veljavnimi podatki – uspešno.
• Registracija z že obstoječim e-poštnim naslovom – napaka (pravilno zavrnjeno).
• Prijava z veljavnimi poverilnicami in preusmeritev na nadzorno ploščo – uspešno.
• Prijava z napačnim geslom (preverjanje BCrypt verifikacije) – napaka (dostop zavrnjen).

Upravljanje knjižnice in opreme:

• Dodajanje nove knjige s sliko in izbiro jezika – uspešno.
• Dodajanje tehnične opreme brez obvezne serijske številke – napaka (validacija deluje).
• Urejanje obstoječih podatkov o zalogi – uspešno.
• Brisanje predmeta, ki je trenutno izposojen – preprečeno (integriteta podatkov).

Izposoje in rezervacije:

• Rezervacija predmeta za prost termin – uspešno.
• Rezervacija predmeta v terminu, ko je ta že zaseden – napaka (pravilno blokirano).
• Vračilo predmeta z vnosom opisa poškodbe – uspešno zabeleženo v bazi.

REST API in varnost:

• Testiranje vseh končnih točk z orodjem Swagger – uspešno.
• Preverjanje pravilnosti JSON odgovorov in statusnih kod (200 OK, 404 Not Found) – uspešno.
• Poskus dostopa do baze preko SQL injekcije v iskalno polje – neuspešno (EF Core zaščita).

4.2 Predstavitev končnega izdelka

Končna aplikacija bo dostopna na naslednjih naslovih (ko bo izveden deployment):

• Frontend (Uporabniški vmesnik): https://knjiznica-frontend.onrender.com/
• Backend (API dokumentacija): https://knjiznica-backend.onrender.com/health/db

Aplikacija omogoča:

• Celovito upravljanje knjižničnega gradiva in tehnične opreme.

• Napreden sistem rezervacij in sledenje izposojam v realnem času.

• Vodenje evidence poškodb s slikovnim gradivom.

• Varno prijavo in upravljanje vlog.

• Dostop do surovih podatkov preko REST API-jev za morebitne zunanje integracije.

Izvorna koda je dostopna na GitHubu:

• Repozitorij: https://github.com/TejVerbicSCV/solska-knji-nica-in-izposoja-tehnicne-opreme

4.3 Rezultati

Rezultat projekta je popolnoma funkcionalna spletna aplikacija, ki izpolnjuje vse zastavljene cilje:

1. Razvila sva zaledni del v .NET 9.0 z REST API-ji.
2. Razvila sva sodoben čelni del v React 19 s poudarkom na uporabniški izkušnji.
3. Implementirala sva relacijsko bazo PostgreSQL z avtomatsko inicializacijo sheme.
4. Zagotovila sva varnost s kriptografskim zgoščevanjem gesel (BCrypt).
5. Pripravila sva aplikacijo za gostovanje na platformi Render.
6. Celotna izvorna koda je ustrezno dokumentirana in objavljena.

Aplikacija je v trenutni obliki pripravljena za uporabo v šolskem okolju in predstavlja odlično osnovo za digitalno transformacijo knjižnice.

Možnosti nadgradenj:

• Avtomatizacija obvestil: Implementacija e-poštnih opominov za zamudnine.

• Skeniranje kod: Integracija čitalca črtnih kod ali QR kod za hitrejšo obdelavo izposoj.

• Statistična analiza: Vizualizacija najbolj izposojenega gradiva z uporabo grafov.

• Mobilna aplikacija: Razvoj mobilne razširitve za dijake (npr. Flutter).

5. ZAKLJUČEK

V okviru zaključne naloge sva uspešno razvila celovito spletno aplikacijo za upravljanje šolske knjižnice in izposojo tehnične opreme. Aplikacija administratorjem omogoča enostavno dodajanje, urejanje in pregledovanje inventarja, dijakom pa hiter vpogled v razpoložljivost gradiva ter oddajo rezervacij.

Projekt je zahteval poznavanje in integracijo različnih sodobnih tehnologij in pristopov – od razvoja zalednega dela v ogrodju **.NET 9.0**, razvoja čelnega dela v knjižnici **React 19**, do upravljanja relacijske podatkovne baze **PostgreSQL** in načrtovanja gostovanja na platformi **Render**.

Posebno pozornost sva namenila varnosti aplikacije in integriteti podatkov. Vsa uporabniška gesla sva šifrirala z algoritmom **BCrypt**, zaščito pred SQL injection napadi pa sva zagotovila z uporabo ogrodja **Entity Framework Core**, ki uporablja parametrizirane poizvedbe.

Med razvojem sva se naučila:

- razvoja zmogljivih **REST API-jev** v ogrodju .NET 9.0 (C#),
- dela z naprednim objektno-relacijskim preslikovanjem preko **Entity Framework Core**,
- razvoja dinamičnih in odzivnih spletnih aplikacij v knjižnici **React 19**,
- uporabe **TypeScript-a** za zagotavljanje varnejše in bolj berljive kode,
- oblikovanja modernih uporabniških vmesnikov s pomočjo **Tailwind CSS 4**,
- verzioniranja kode in sodelovanja pri projektu z uporabo orodij **Git** in **GitHub**.

Aplikacija je v celoti funkcionalna in uporabna v realnem šolskem okolju, hkrati pa predstavlja odlično osnovo za nadaljnji razvoj. V prihodnosti bi jo lahko nadgradila z dodatnimi funkcionalnostmi, kot so avtomatska e-poštna obvestila, integracija skenerja črtnih kod ter napredna statistična obdelava podatkov o izposojah.

Zaključna naloga nama je omogočila, da sva pridobila dragocene praktične izkušnje z razvojnim ciklom spletne aplikacije od začetka do konca – od prvega načrtovanja sheme baze podatkov, preko programiranja poslovne logike, do končnega dokumentiranja celotnega

sistema. Podatki in znanja, ki sva jih pridobila, nama bodo v veliko pomoč pri nadaljnji poklicni poti na področju računalništva.

5.1 Samoevalvacija

Pri izvajanju najine zaključne naloge sva prišla do naslednjih spoznanj in ocen najinega dela:

Doseganje ciljev: Meniva, da sva uspešno dosegla vse zastavljene cilje. Aplikacija ni le delujoč prototip, temveč stabilna rešitev, ki vključuje vse ključne funkcionalnosti: od varne avtentikacije do kompleksnega sistema za upravljanje s poškodbami opreme.

Tehnični izzivi in rešitve: Največji tehnični izziv nama je predstavljala logika rezervacij. Preprečevanje prekrivanja terminov izposoje na nivoju baze in strežnika je zahtevalo natančno načrtovanje SQL poizvedb in validacij v .NET kontrolerjih. Prav tako sva se na začetku soočala s težavami pri nastavljanju CORS politike, kar sva rešila s pravilno konfiguracijo v datoteki Program.cs. Ti izzivi so nama pomagali poglobiti znanje o delovanju spletnih protokolov.

Organizacija dela in sodelovanje: Delo sva si razdelila enakomerno, glede na najina močna področja. . Kljub delitvi dela sva redno sodelovala pri načrtovanju strukture API-jev, kar je zagotovilo, da sta se oba dela aplikacije brez težav povezala. Verzioniranje s sistemom Git nama je omogočilo, da sva napake odpravljala sproti, ne da bi motila delo drug drugega.

Kaj bi spremenila? Če bi imela na voljo več časa, bi v projekt vključila več avtomatiziranih testov (Unit Tests), da bi še dodatno zmanjšala možnost napak pri prihodnjih nadgradnjah. Prav tako bi raziskala možnost uporabe Docker vsebnikov že v zgodnji fazi razvoja, kar bi še poenostavilo končni prenos na produkcijski strežnik.

Zaključna misel: Projekt nama je dal dragocen vpogled v to, kako poteka razvoj programske opreme v realnem svetu. Pridobila sva samozavest pri uporabi sodobnih orodij, kot sta .NET 9 in React 19, ter se naučila, kako pomembno je natančno načrtovanje pred samim pisanjem kode. Verjameva, da je najina aplikacija uporabna in bi jo z minimalnimi prilagoditvami lahko uporabili v katerikoli srednji šoli.